Core Knowledge Series: **Hands-On with Python**

Session One: **Introduction and Data Manipulation**

Bioinformatics Core (BSR)

CSH Cold Spring Harbor Laboratory

Jupyter and Python

- Python is a versatile open-source programming language widely used for statistical computing, data analysis, and visualization.
- Jupyter Notebooks provide an interface that makes Python coding easier with features like inline plotting, code annotation, and real-time output.

Python vs R

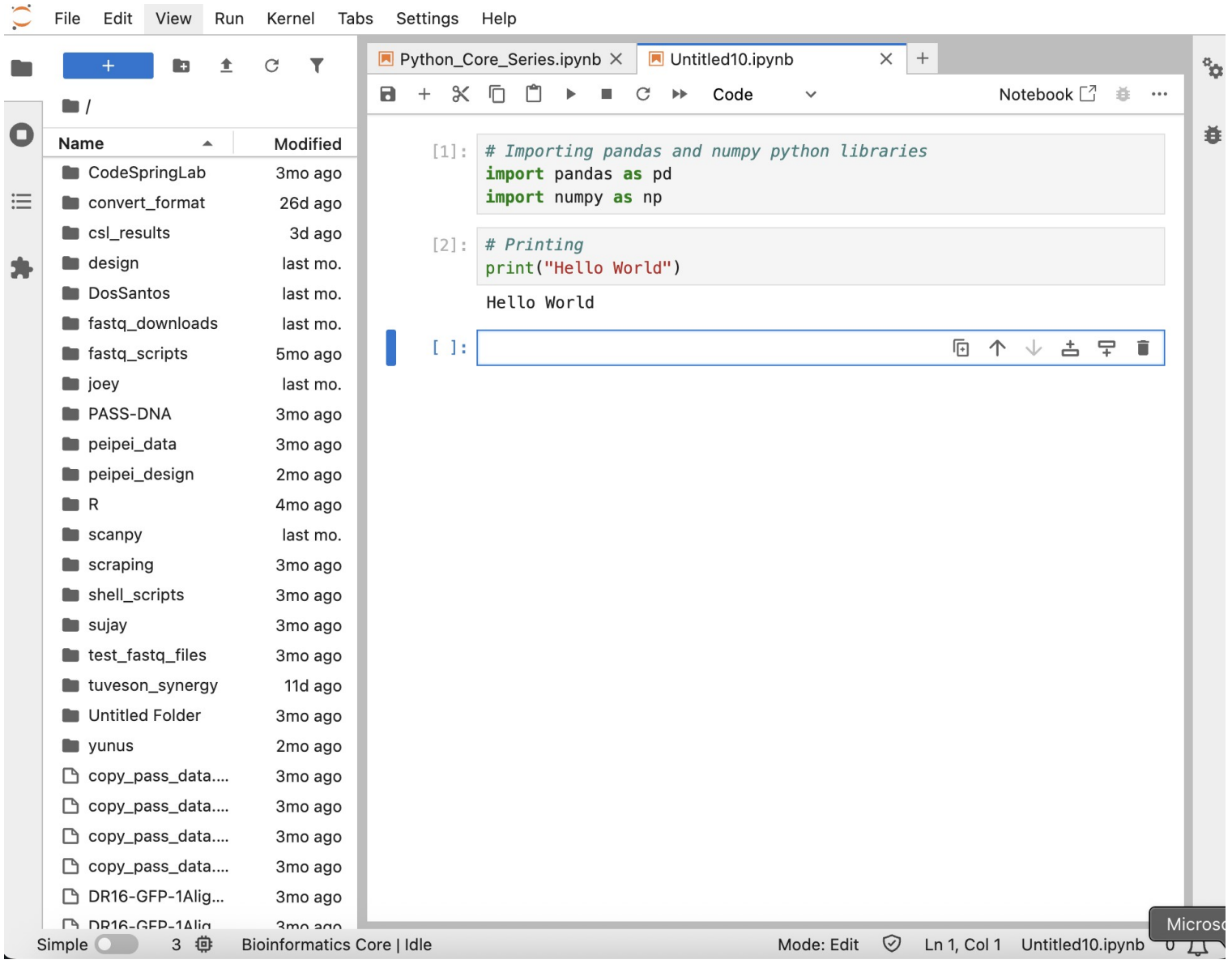
	Python	R
Purpose	**Statistics, data manipulation/visualization** **General purpose** – Machine learning, web development	Specifically built for **statistics, data manipulation/visualization**
Index	**0-based indexing**, counting starts at 0 (first item is index 0)	**1-based indexing**, counting starts at 1 (first item is index 1).
Libraries	**scanpy** (single-cell), **pandas, numpy** (data manipulation)	**DeSeq2** (Differential Expression), **Seurat** (single-cell), **GSEA** (pathway analysis)
Interface	**Jupyter** (for beginners) **PyCharm, spyder, VSCode**	**RStudio**

Jupyter Lab vs Jupyter Hub vs Jupyter Notebook

- Jupyter Lab is a python interface, similar to Rstudio for R
- Jupyter Hub is like jupyter lab, but connected to a server. In this case, Elzar
- Jupyter notebooks are individual scripts where python code is run

CSH Cold Spring Harbor Laboratory

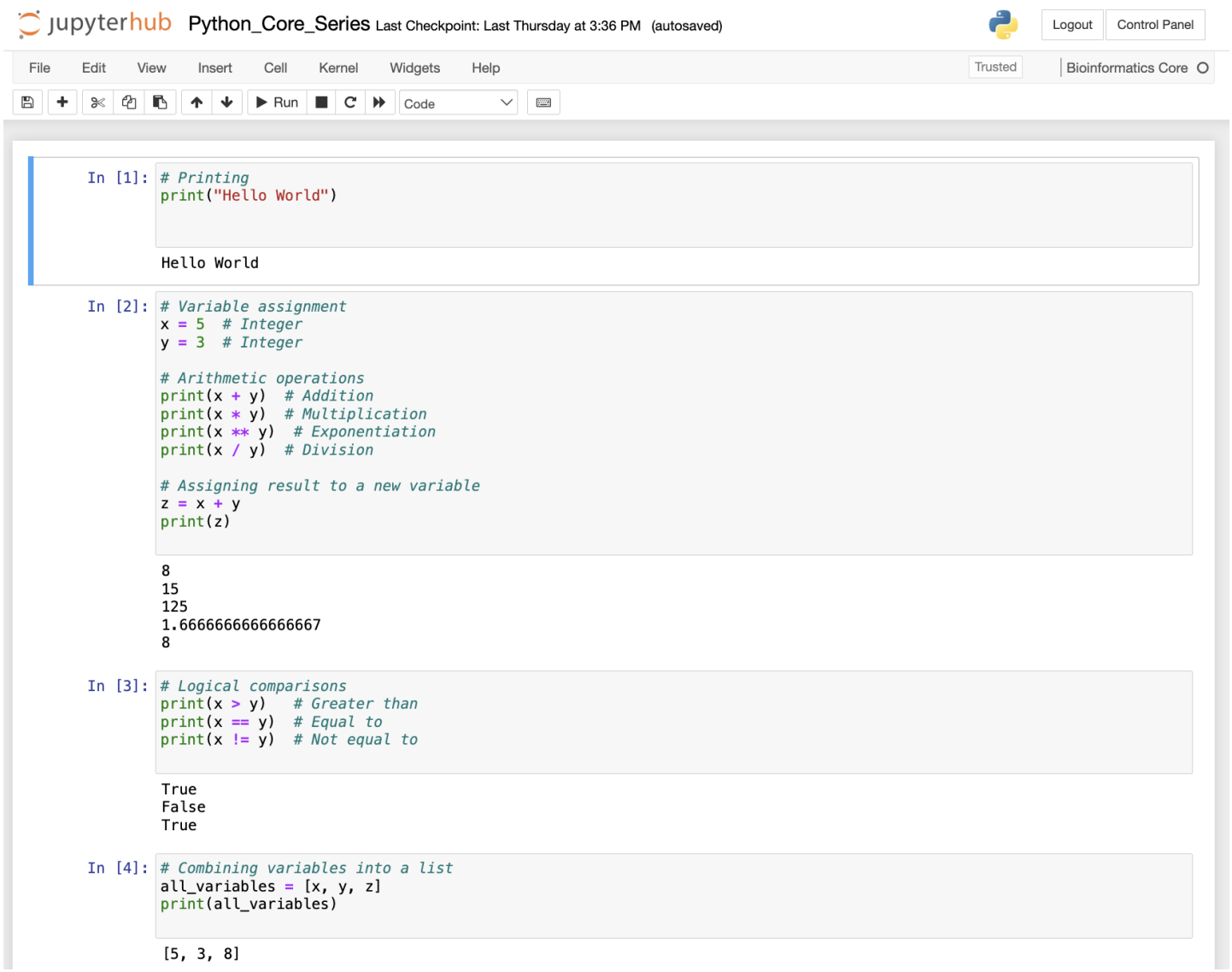
Jupyter Introduction



- Logging in
  - https://bamdev4.cshl.edu/jupyter
- Home directory
  - Jupyter hub on Elzar will open in your home directory
- Creating files
  - ”+” in the top left used to create files and folders

Cold Spring Harbor Laboratory

Jupyter Notebooks



- Chunk Number
  - In square brackets on left
- Code chunk
  - Contains python code, all run at the same time
- Output
  - Jupyter outputs below each chunk

Cold Spring Harbor Laboratory

Basic Python Commands

In [1]:
```
# Printing
print("Hello World")
```

```
Hello World
```

In [2]:
```
# Variable assignment
x = 5  # Integer
y = 3  # Integer

# Arithmetic operations
print(x + y)  # Addition
print(x * y)  # Multiplication
print(x ** y)  # Exponentiation
print(x / y)  # Division

# Assigning result to a new variable
z = x + y
print(z)
```

```
8
15
125
1.6666666666666667
8
```

In [3]:
```
# Logical comparisons
print(x > y)   # Greater than
print(x == y)  # Equal to
print(x != y)  # Not equal to
```

```
True
False
True
```

In [4]:
```
# Combining variables into a list
all_variables = [x, y, z]
print(all_variables)
```

```
[5, 3, 8]
```

- **print** function
- To run a chunk of code, move cursor to specific chunk, hit "Run" on the top menu bar
- Variable assignment with "="
- Performing arithmetic operations
- Assigning variables to variables
- Logical comparisons – what do they return/print?

Cold Spring Harbor Laboratory

Documentation for functions

```
[12]: print?

Docstring:
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

Prints the values to a stream, or to sys.stdout by default.
Optional keyword arguments:
file:  a file-like object (stream); defaults to the current sys.stdout.
sep:   string inserted between values, default a space.
end:   string appended after the last value, default a newline.
flush: whether to forcibly flush the stream.
Type:      builtin_function_or_method
```

- Using ? after a function will provide useful documentation for that function.
  - Example: print?

Cold Spring Harbor Laboratory

Practice

- Create variables a and b
  - a is 7
  - b is 9
- c is the product of a and b
- d is a to the power of b
- e is c minus d
- Store a,b,c,d,e in a list called all_numbers and print

Practice Solution

[1]:
```
# Create variables
a = 7
b = 9

# Perform calculations
c = a * b
d = a ** b
e = c - d

# Store all values in a vector (list)
all_numbers = [a, b, c, d, e]

# Optional: print the result
print(all_numbers)
```

```
[7, 9, 63, 40353607, -40353544]
```

Cold Spring Harbor Laboratory

Types of variables in Python

```
In [10]: # Different types of variables
int_var = 10                    # Integer
num_var = 10.5                  # Float (numeric)
char_var = "Hello"              # String (character)
bool_var = True                 # Boolean (logical)
complex_var = 3 + 4j            # Complex number
list_var = [1, 2, 3]            # List
tuple_var = (4, 5, 6)           # Tuple
set_var = {7, 8, 9}             # Set
dict_var = {"a": 1, "b": 2}     # Dictionary
none_var = None                 # NoneType

# Checking variable type
print(type(int_var))      # <class 'int'>
print(type(num_var))      # <class 'float'>
print(type(char_var))     # <class 'str'>
print(type(bool_var))     # <class 'bool'>
print(type(complex_var))  # <class 'complex'>
print(type(list_var))     # <class 'list'>
print(type(tuple_var))    # <class 'tuple'>
print(type(set_var))      # <class 'set'>
print(type(dict_var))     # <class 'dict'>
print(type(none_var))     # <class 'NoneType'>
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'complex'>
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
<class 'NoneType'>
```

- Numeric variables
  - Integers – Whole numbers. specifies integer.
  - Float – Real numbers with decimal point
- Character/String variables
  - Text or string data
- Logical/Boolean variables
  - TRUE or FALSE
- Data structures
  - List – Ordered, mutable
  - Tuple – Ordered, immutable
  - Set – Unique, unordered, typically used for unions, intersections

Numpy and Arrays

In [11]:
```
# Creating a NumPy array (numeric vector)
vec_num = np.array([1, 2, 3, 4, 5])
print(vec_num)

# NumPy arrays support vectorized operations
print(vec_num + 10)    # Add 10 to each element
print(vec_num * 2)     # Multiply each element by 2
print(vec_num > 3)     # Logical comparison

# Indexing works similarly to Python lists
print(vec_num[0])      # First element
print(vec_num[-1])     # Last element

# Slicing
print(vec_num[1:4])    # Elements from index 1 to 3 (Python is 0-based)

# Type and shape
print(vec_num.shape)   # (5,)
```

```
[1 2 3 4 5]
[11 12 13 14 15]
[ 2  4  6  8 10]
[False False False  True  True]
1
5
[2 3 4]
(5,)
```

- **Numpy** is a tool for mathematical computing
  - Supports multi-dimensional array/matrix operations
  - * FASTER MATHEMATICAL OPERATIONS THAN LISTS
  - Good for big data due to speed

Numpy arrays are often easier to work with than lists

```
In [12]: # Python list
         py_list = [1, 2, 3, 4, 5]

         # Creating empty list
         add_ten = []

         # Adding 10 to each element and appending to the empty list
         for x in py_list:
             add_ten.append(x+10)


         print(add_ten)

         # With NumPy array (vectorized)
         np_array = np.array(py_list)
         print(np_array + 10)
```

```
[11, 12, 13, 14, 15]
[11 12 13 14 15]
```

- Mathematical operations are easier to perform on arrays
- Must be done manually with lists

Cold Spring Harbor Laboratory

Pandas

```
In [13]: # Create the DataFrame
         count_data = pd.DataFrame({
             "Gene": ["Gene1", "Gene2"],
             "Sample1": [100, 200],
             "Sample2": [150, 250]
         })

         # Display the DataFrame
         count_data
```

Out[13]:

	Gene	Sample1	Sample2
0	Gene1	100	150
1	Gene2	200	250

- **Pandas** is a tool for mathematical computing
  - Data Frame creation (most used way to store 2D data)
  - Data frames are created using a dictionary
  - A dictionary is a data type used to store key-value pairs
    - Key: Column name
    - Value: Column values
  - Pandas contains functions for data manipulation and cleaning with data frames

Practice Question

- Create a data frame in pandas with 3 columns
  - Gene: “TP53” and “BRCA1”
  - Chromosome: “17” and “17”
  - Gene_ID: “ENSG00000141510” and ”ENSG00000012048”

Cold Spring Harbor Laboratory

Practice Question Solution

In [1]:
```
import pandas as pd
gene_data = pd.DataFrame({
    "Gene": ["TP53", "BRCA1"],
    "Chromosome": ["17", "17"],
    "Gene_ID": ["ENSG00000141510", "ENSG00000012048"]
})
```

In [2]:
```
gene_data
```

Out[2]:

	Gene	Chromosome	Gene_ID
0	TP53	17	ENSG00000141510
1	BRCA1	17	ENSG00000012048

Cold Spring Harbor Laboratory

Reading in example counts data

In [14]:
```
# Step 1: Read RNA-seq raw counts CSV from GitHub
url = 'https://raw.githubusercontent.com/jimmyrouse/BSR/main/Tumor_vs_Normal_counts.csv'
df_counts = pd.read_csv(url)
df_counts.head()
```

Out[14]:

	Gene	Tumor_1	Tumor_2	Tumor_3	Normal_1	Normal_2	Normal_3
0	TP53	92	114	111	89	100	104
1	EGFR	107	94	111	85	80	100
2	MYC	101	101	114	89	82	97
3	KRAS	130	100	124	69	68	90
4	PIK3CA	106	98	92	99	99	111

- read_csv function will read in a csv file in python
- head will show the first few rows of the data frame
- This data frame has 1 column identifying the gene and 6 columns with sample identifiers.

Cold Spring Harbor Laboratory

Should the gene name be a column?

```
In [15]: # Step 2: Set the 'Gene' column as the index
df_counts.set_index('Gene', inplace=True)

df_counts.head()
```

Out[15]:

Gene	Tumor_1	Tumor_2	Tumor_3	Normal_1	Normal_2	Normal_3
TP53	92	114	111	89	100	104
EGFR	107	94	111	85	80	100
MYC	101	101	114	89	82	97
KRAS	130	100	124	69	68	90
PIK3CA	106	98	92	99	99	111

- Data frames can have indices, similar to row names in R.
- The set_index method will set a column as the indices of a data frame.
- inplace argument specifies to change the object directly
  - No need to assign to df_counts

Cold Spring Harbor Laboratory

Functions vs Methods

```
In [15]: # Step 2: Set the 'Gene' column as the index
df_counts.set_index('Gene', inplace=True)

df_counts.head()
```

Out[15]:

Gene	Tumor_1	Tumor_2	Tumor_3	Normal_1	Normal_2	Normal_3
TP53	92	114	111	89	100	104
EGFR	107	94	111	85	80	100
MYC	101	101	114	89	82	97
KRAS	130	100	124	69	68	90
PIK3CA	106	98	92	99	99	111

- Function – Independent, not tied to any object
  - Example: print()
- Method – associated with an object.
  - Example: .head() and .set_index() belong to the DataFrame object.
- Syntax:
  - Function: print(df_counts)
  - Method: df_counts.set_index()

Practice Question

- Get help for the .set_index() method
  - Hint: You will need to include the object here (df_counts), as the method is tied to the object

Practice Solution

```
[14]: df_counts.set_index?
```

```
Signature:
df_counts.set_index(
    keys,
    drop: 'bool' = True,
    append: 'bool' = False,
    inplace: 'bool' = False,
    verify_integrity: 'bool' = False,
)
Docstring:
Set the DataFrame index using existing columns.

Set the DataFrame index (row labels) using one or more existing
columns or arrays (of the correct length). The index can replace the
existing index or expand on it.

Parameters
----------
keys : label or array-like or list of labels/arrays
    This parameter can be either a single column key, a single array of
    the same length as the calling DataFrame, or a list containing an
    arbitrary combination of column keys and arrays. Here, "array"
    encompasses :class:`Series`, :class:`Index`, ``np.ndarray``, and
    instances of :class:`~collections.abc.Iterator`.
drop : bool, default True
    Delete columns to be used as the new index.
append : bool, default False
    Whether to append columns to existing index.
inplace : bool, default False
    If True, modifies the DataFrame in place (do not create a new object).
verify_integrity : bool, default False
    Check the new index for duplicates. Otherwise defer the check until
    necessary. Setting to False will improve the performance of this
    method.

Returns
-------
DataFrame or None
    Changed row labels or None if ``inplace=True``.
```

Cold Spring Harbor Laboratory

Performing CPM normalization on raw counts

In [16]:
```
# Step 3: CPM Normalization
# Calculate column sums (total counts per sample)
#axis=0 calcualtes the sum DOWN the columns
total_counts = df_counts.sum(axis=0)

total_counts
```

Out[16]:
```
Tumor_1     4903
Tumor_2     4993
Tumor_3     4886
Normal_1    5043
Normal_2    4860
Normal_3    5021
dtype: int64
```

In [17]:
```
# Divide each value by the column's total count, then multiply by 1e6
df_cpm = df_counts.div(total_counts, axis=1) * 1e6

# Step 4: Save the CPM-normalized data to a CSV
#df_cpm.to_csv("Tumor_vs_Normal_CPM.csv")

# Preview
print("CPM-normalized data:")
df_cpm.head()
```

```
CPM-normalized data:
```

Out[17]:

Gene	Tumor_1	Tumor_2	Tumor_3	Normal_1	Normal_2	Normal_3
TP53	18764.022027	22831.964751	22717.969709	17648.225263	20576.131687	20713.005377
EGFR	21823.373445	18826.356900	22717.969709	16855.046599	16460.905350	19916.351324
MYC	20599.632878	20228.319648	23331.968891	17648.225263	16872.427984	19318.860785
KRAS	26514.378952	20028.039255	25378.632828	13682.331945	13991.769547	17924.716192
PIK3CA	21619.416684	19627.478470	18829.308228	19631.171921	20370.370370	22107.149970

- Let's do some calculations
- **Counts per million** helps to normalize for sample sequencing depth.
- Calculation:
  - calculate the total counts for each sample
  - divide raw counts by total counts
  - multiply by 1 million
- .sum and .div are methods applied to all rows or columns of a data frame
- axis=0 - perform the addition down each column,
- axis=1 perform the division across each column

Cold Spring Harbor Laboratory

Data frame indexing

```
In [18]: # First column (all rows)
         col1 = df_cpm.iloc[:, 0]
         print("First column (all rows):")
         col1.head(10)
```

```
In [19]: # First row (all columns)
         row1 = df_cpm.iloc[0, :]
         print("First row (all columns):")
         row1
```

```
In [20]: # Columns 1 to 3 (all rows)
         cols_1_to_3 = df_cpm.iloc[:, 0:3]
         print("Columns 1 to 3")
         cols_1_to_3.head()
```

```
Columns 1 to 3
```

```
In [21]: # Columns 4 to 6 (all rows)
         cols_4_to_6 = df_cpm.iloc[:, 3:6]
         print("Columns 4 to 6 (all rows):")
         cols_4_to_6.head()
```

```
Columns 4 to 6 (all rows):
```

```
In [22]: # Single value at first row, first column
         value_0_0 = df_cpm.iloc[0, 0]
         print("Single value at first row, first column:")
         print(value_0_0)
```

- Columns and rows can be reached using .iloc and brackets after the data frame [,]
- Comma separates row index from column index with row index coming first
- Python is 0-indexed, meaning that the first element is at index 0
- : Used to slice. If left blank, it will use all rows/columns. 3:6 specifies the 4th to 6th column.

Cold Spring Harbor Laboratory

Practice

- f is the value in the first row and second column of df_cpm dataframe
- g is the value in the second row and third column of df_cpm dataframe
- h is f divided by g
- i is the first column of df_cpm
- j is the first column of df_cpm multiplied by h

Practice Solution

```
import pandas as pd

f = df_cpm.iloc[0, 1]
g = df_cpm.iloc[1, 2]
h = f / g
i = df_cpm.iloc[:, 0]
j = i * h

print(f)
print(g)
print(h)
print(i.head())
print(j.head())
```

Pandas

- pandas is a Python library for fast and easy data analysis and manipulation.
- Uses DataFrames to handle tabular data (like Excel or SQL).
- Great for cleaning, filtering, summarizing, and transforming datasets.

Simple pandas data frame methods

In [34]:
```
####################PANDAS ON CPM MATRIX##############

# Overview of the DataFrame
df_cpm.info()

# Summary statistics
df_cpm.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 50 entries, TP53 to PRKAR1A
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Tumor_1   50 non-null     float64
 1   Tumor_2   50 non-null     float64
 2   Tumor_3   50 non-null     float64
 3   Normal_1  50 non-null     float64
 4   Normal_2  50 non-null     float64
 5   Normal_3  50 non-null     float64
dtypes: float64(6)
memory usage: 2.7+ KB
```

Out[34]:

	Tumor_1	Tumor_2	Tumor_3	Normal_1	Normal_2	Normal_3
count	50.000000	50.000000	50.000000	50.000000	50.000000	50.000000
mean	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000
std	2540.548095	2855.423301	3081.263902	2857.322660	3266.095107	2639.778020
min	15704.670610	13819.347086	12075.317233	13682.331945	13786.008230	15733.917546
25%	18356.108505	17875.025035	18215.309046	17697.798929	17952.674897	17974.507070
50%	19885.784214	19827.758862	20466.639378	19829.466587	19444.444444	20015.933081
75%	21517.438303	21430.002003	22666.803111	22010.707912	22222.222222	21509.659430
max	26514.378952	27238.133387	25378.632828	25778.306564	26954.732510	28480.382394

- .info() gives information about the columns in a data frame
- .describe() will show count, mean, std, min, max for each column.

Cold Spring Harbor Laboratory

Pandas on RNA-seq data

- pandas has functions where we can annotate log fold change, p-value, etc. for certain genes

Cold Spring Harbor Laboratory

Calculating tumor and normal averages

In [31]:
```
# Calculate Normal mean across first 3 columns
df_cpm['Normal_Mean'] = df_cpm.iloc[:, 3:6].mean(axis=1)

# Calculate Tumor mean across next 3 columns (adjust if needed!)
df_cpm['Tumor_Mean'] = df_cpm.iloc[:, 0:3].mean(axis=1)

# Calculate difference (Tumor - Normal)
df_cpm['Difference'] = df_cpm['Tumor_Mean'] - df_cpm['Normal_Mean']

# View the updated DataFrame
df_cpm[['Normal_Mean', 'Tumor_Mean', 'Difference']].head()
```

Out[31]:

Gene	Normal_Mean	Tumor_Mean	Difference
TP53	19645.787442	21437.985496	1792.198053
EGFR	17744.101091	21122.566685	3378.465593
MYC	17946.504677	21386.640472	3440.135795
KRAS	15199.605895	23973.683678	8774.077784
PIK3CA	20702.897421	20025.401127	-677.496294

- Adding a new column to a data frame
  - df_cpm[“New_column_name”]
- We can use .iloc to select for tumor samples and normal samples
- .mean(axis=1) will average across the columns (each gene)
- We can then subtract these two columns to get the mean difference

Cold Spring Harbor Laboratory

Ranking and sorting by mean difference

In [32]:
```
# Add a ranking column based on Difference
df_cpm['Rank'] = df_cpm['Difference'].rank(ascending=False)

# View top 5 upregulated genes
df_cpm[['Normal_Mean', 'Tumor_Mean', 'Difference', 'Rank']].sort_values(by='Difference', ascending=False).head()
```

Out[32]:

Gene	Normal_Mean	Tumor_Mean	Difference	Rank
KRAS	15199.605895	23973.683678	8774.077784	1.0
MAPK1	16826.536549	24085.661552	7259.125003	2.0
AKT1	18170.674685	24676.462536	6505.787850	3.0
PTEN	16472.604682	22519.670343	6047.065661	4.0
CTNNB1	17201.712434	23192.452756	5990.740322	5.0

- .rank() will rank a column in a data frame
- .sort_values() will sort the data frame by a certain column

Practice Question

- Sort and rank a DataFrame by (Normal - Tumor) mean in order so that genes where Normal is higher than Tumor appear at the top?
- Output just columns Difference and Rank

Practice Question Solution

In [44]:
```
# Add a ranking column based on Difference
df_cpm['Rank'] = df_cpm['Difference'].rank(ascending=True)

# View top 5 upregulated genes
df_cpm[['Difference', 'Rank']].sort_values(by='Difference', ascending=True).head()
```

Out[44]:

Gene	Difference	Rank
MTOR	-9404.481237	1.0
CDK4	-9202.129705	2.0
SMARCB1	-8201.473494	3.0
FOS	-6917.679046	4.0
ABL1	-6208.915850	5.0

Calculating log2 fold change

In [45]:
```
# Calculate fold change and log2 fold change
df_cpm['Fold_Change'] = (df_cpm['Tumor_Mean'] + 1) / (df_cpm['Normal_Mean'] + 1)
df_cpm['Log2_FC'] = np.log2(df_cpm['Fold_Change'])

# View top genes with highest log2 fold change
df_cpm[['Normal_Mean', 'Tumor_Mean','Fold_Change', 'Log2_FC']].sort_values(by='Log2_FC', ascending=False).head()
```

Out[45]:

Gene	Normal_Mean	Tumor_Mean	Fold_Change	Log2_FC
KRAS	15199.605895	23973.683678	1.577219	0.657383
MAPK1	16826.536549	24085.661552	1.431384	0.517410
PTEN	16472.604682	22519.670343	1.367076	0.451093
AKT1	18170.674685	24676.462536	1.358018	0.441503
CTNNB1	17201.712434	23192.452756	1.348244	0.431082

- We can divide the two columns to get the fold change
- Add 1 to the counts to avoid dividing by zero
- log2 attribute of numpy will calculate the log2 fold change

Cold Spring Harbor Laboratory

Calculating p-value between tumor and normal samples for each gene

In [46]:
```
# Importing library to perform t-test
from scipy.stats import ttest_ind
```

In [48]:
```
# Creating data frames of normal and tumor samples
normal_vals = df_cpm.iloc[:, 3:6]
tumor_vals = df_cpm.iloc[:, 0:3]

normal_vals.head()
```

Out[48]:

Gene	Normal_1	Normal_2	Normal_3
TP53	17648.225263	20576.131687	20713.005377
EGFR	16855.046599	16460.905350	19916.351324
MYC	17648.225263	16872.427984	19318.860785
KRAS	13682.331945	13991.769547	17924.716192
PIK3CA	19631.171921	20370.370370	22107.149970

In [49]:
```
# Apply t-test row-wise
ttest_results = ttest_ind(tumor_vals.values, normal_vals.values, axis=1, equal_var=False)

# Add p-values to dataframe
df_cpm['p_value'] = ttest_results.pvalue

df_cpm[['Normal_Mean', 'Tumor_Mean','Fold_Change', 'Log2_FC', 'p_value']].sort_values(by='Log2_FC', ascending=False)
```

Out[49]:

Gene	Normal_Mean	Tumor_Mean	Fold_Change	Log2_FC	p_value
KRAS	15199.605895	23973.683678	1.577219	0.657383	0.027548
MAPK1	16826.536549	24085.661552	1.431384	0.517410	0.000128
PTEN	16472.604682	22519.670343	1.367076	0.451093	0.002597
AKT1	18170.674685	24676.462536	1.358018	0.441503	0.080380

- scipy.stats has the function to perform a t-test
- T-test will measure the significance of the difference between tumor vs normal counts for each gene
- SWAP tumor_vals and normal_vals

Cold Spring Harbor Laboratory

Assigning downregulated and upregulated labels to genes based on p-value

In [41]:
```
## By default the label column will be not significant
df_cpm['Label'] = 'Not Significant'

# Define significance thresholds
p_thresh = 0.05     # Adjusted p-value

## Classify gene as upregulated of Log2 FC is greater than 0 and p-value is significant
## Classify gene as downregulated of Log2 FC is less than 0 and p-value is significant

df_cpm.loc[(df_cpm['Log2_FC'] > 0) & (df_cpm['p_value'] < p_thresh), 'Label'] = 'Upregulated'
df_cpm.loc[(df_cpm['Log2_FC'] < 0) & (df_cpm['p_value'] < p_thresh), 'Label'] = 'Downregulated'

# Count how many upregulated and downregulated genes there are
df_cpm['Label'].value_counts()
```

Out[41]:
```
Not Significant    36
Upregulated         8
Downregulated       6
Name: Label, dtype: int64
```

- Create a new column called label
- By default, each gene is "Not significant"
- If the log fold change is positive and p-value is significant, the gene is "Upregulated"
- If the log fold change is negative and p-value is significant, the gene is "Upregulated"

Assigning downregulated and upregulated labels to genes based on p-value

In [50]:
```
## By default the label column will be not significant
df_cpm['Label'] = 'Not Significant'

# Define significance thresholds
p_thresh = 0.05     # Adjusted p-value

## Classify gene as upregulated of Log2 FC is greater than 0 and p-value is significant
## Classify gene as downregulated of Log2 FC is less than 0 and p-value is significant

df_cpm.loc[(df_cpm['Log2_FC'] > 0) & (df_cpm['p_value'] < p_thresh), 'Label'] = 'Upregulated'
df_cpm.loc[(df_cpm['Log2_FC'] < 0) & (df_cpm['p_value'] < p_thresh), 'Label'] = 'Downregulated'

# Count how many upregulated and downregulated genes there are
df_cpm['Label'].value_counts()
```

Out[50]:
```
Not Significant    36
Downregulated       8
Upregulated         6
Name: Label, dtype: int64
```

- value_counts counts the number of times a unique value appears in a column
- We now have a data frame with these columns
  - P-value
  - Log2fold change
  - Mean of normal and tumor counts
  - Whether the gene is significantly upregulated/downregulated

Joining data

In [42]:
```
##### JOINING METADATA

# Meta data 1
meta_data_1 = pd.DataFrame({
    "Sample": ["Normal_1", "Normal_2", "Normal_3", "Tumor_1"],
    "Group": ["Normal", "Normal", "Normal", "Tumor"],
    "Condition": ["Healthy", "Healthy", "Healthy", "Cancer"]
})

# Meta data 2
meta_data_2 = pd.DataFrame({
    "Sample": ["Normal_2", "Tumor_1", "Tumor_2", "Tumor_3"],
    "Batch": ["A", "B", "B", "C"]
})
```

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

Left joining

```
# Left join: keep all rows from meta_data_1
left_join_result = meta_data_1.merge(meta_data_2, on="Sample", how="left")
```

meta_data_1

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

meta_data_2

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

left_join_result

	Sample	Group	Condition	Batch
1	Normal_1	Normal	Healthy	*NA*
2	Normal_2	Normal	Healthy	A
3	Normal_3	Normal	Healthy	*NA*
4	Tumor_1	Tumor	Cancer	B

- Left joining merges two data frames
  - keeps all rows from the first data frame
  - matching the two data frames by specific column(s)
  - adding columns from second data frame to the first data frame
  - Will return NA for samples where there is no match

Full/Outer joining

```
# Full join: union of both datasets
full_join_result = meta_data_1.merge(meta_data_2, on="Sample", how="outer")
```

meta_data_1

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

meta_data_2

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

full_join_result

	Sample	Group	Condition	Batch
1	Normal_1	Normal	Healthy	*NA*
2	Normal_2	Normal	Healthy	A
3	Normal_3	Normal	Healthy	*NA*
4	Tumor_1	Tumor	Cancer	B
5	Tumor_2	*NA*	*NA*	B
6	Tumor_3	*NA*	*NA*	C

- Full joining merges two data frames
  - keeps all samples from both data frames
  - matching the two data frames by specific column(s)
  - adding columns from second data frame to the first data frame
  - Will return NA for samples where there is no match

Inner joining

```
# Inner join: intersection (only matching "Sample")
inner_join_result = meta_data_1.merge(meta_data_2, on="Sample", how="inner")
```

meta_data_1

	Sample	Group	Condition
1	Normal_1	Normal	Healthy
2	Normal_2	Normal	Healthy
3	Normal_3	Normal	Healthy
4	Tumor_1	Tumor	Cancer

meta_data_2

	Sample	Batch
1	Normal_2	A
2	Tumor_1	B
3	Tumor_2	B
4	Tumor_3	C

inner_join_result

	Sample	Group	Condition	Batch
1	Normal_2	Normal	Healthy	A
2	Tumor_1	Tumor	Cancer	B

- Inner joining merges two data frames
  - keeps only samples that are present in both data frames
  - matching the two data frames by specific column(s)
  - adding columns from second data frame to the first data frame

Cold Spring Harbor Laboratory

Resources

BiTS
Bioinformatics Social

Join us !

- Connect with CSHL bioinformaticians
- Share analysis experiences
- Regular best-practice guidelines
- Explore novel tools & pipelines
- GitHub & Teams community

CSH Cold Spring Harbor Laboratory